Find the Promos

Which problem I picked and why

I picked Problem 1 because I could check every suggested cut against a timestamped screenplay. I wanted to give an editor a short list of cuts to review instead of starting with all 50 episodes.

My assumptions

I look for 20 to 60 seconds of footage with a hook in the first three seconds and an open question at the end. Timecodes are relative to each episode. I allow joins between adjacent episodes when a speaking character appears in both clips.

How my tool works

Python reads the timed dialogue and action, finds possible cuts, and rejects boundaries inside spoken lines. The ranker scores the hook, ending and standalone clarity. It gives the most weight to the hook and cliffhanger. It caps overlap at 45% of the shorter cut and allows at most two promos from one episode. Repeated speakers and hook types lose points.

In model mode, the first pass scores a shortlist with episode context. A second pass sees only the clip and checks whether a new viewer could follow it. The code matches the cited lines and timecodes to the screenplay. The editor makes the final call using the ranked list and review sheet.

How it scales to every show

I shortlist up to 12 cuts, send four per request and cache valid responses. Both passes take up to six requests before retries. With a quota of 20 requests per day, I could plan for three shows a day or 21 a week.

At an assumed 25,000 input and 3,000 output tokens per show, the paid estimate is about $0.011 per show, or $0.00022 per episode across 50 episodes. I did not measure live API usage. With a larger budget, I would test a low-cost model on a wider shortlist and reserve a stronger model for the final few cuts.

Results and limitations

How I checked quality

The offline run scored 103,263 candidate cuts across 50 episodes in about 63 seconds. It returned 15 promos, and all 15 passed nine automated checks. The checks included duration, cut boundaries, overlap and whether the cited lines appeared at the reported times. Codex also read the top ten against the scripts to judge whether the opening and ending worked as a story.

Episode 34 (01:34.20 to 02:06.40) is a cut I would show first. The witnesses explain the accusation, and the scene cuts before the accused answers the judge.

Where it fails

Episode 27 (00:44.50 to 01:32.40) opens with a question about a husband. The doors burst open at 00:48.80, more than four seconds into the cut. In Episode 31, the scorer reads a parting insult as a cliffhanger because it ends with a question. I generated 7,011 adjacent-episode joins, but none made the final 15. A shared character can connect two clips while the focus of the scene still changes.

Tools I used, and one place the AI got it wrong

I used Codex to build and check the Python and Streamlit prototype. The first scorer saw “die” in Episode 38 at 00:17.50 and called it a threat. The guard was offering bread to a starving prisoner. Codex revised the rule. I had no Gemini key, so the model flow was tested with a stand-in provider.

What I would do next with more time

I would ask an editor to watch the top cuts against the footage and mark when each hook actually lands. I would then compare those picks with the offline and live model rankings.